

OCR 与 ECC Rust 代码评审规则对比报告

Open Code Review vs everything-claude-code

Rust 审查规则差异与优劣势分析

报告日期: 2026-06-28

对比对象:

OCR — rust.md, cargo_toml.md

ECC — rust-reviewer.md, rust-review.md, rust-patterns

仓库:

github.com/alibaba/open-code-review

github.com/affaan-m/everything-claude-code

一、执行摘要

本报告对比 Open Code Review (OCR) 与 everything-claude-code (ECC) 两套体系中针对 Rust 代码的评审规则。OCR 将规则作为 prompt 注入统一审查流水线；ECC 通过专用 subagent，在运行 cargo 工具链后再进行语义审查。

核心结论：OCR 在 Rust 语义深度 (unsafe、async cancellation、FFI、API 设计) 上更强；ECC 在工程门禁 (check/clippy/fmt/test/audit)、严重级别划分与团队规范上更完整。最佳实践是将 ECC 的工具链门禁与 OCR 的语义规则结合使用。

二、规则来源与架构差异

- 规则文件：OCR=rust.md+cargo_toml.md；ECC=rust-reviewer+rust-review+rust-patterns
- 触发方式：OCR 审查 *.rs 自动匹配；ECC 手动 /rust-review
- 工具链门禁：OCR 无强制 cargo；ECC 强制 check/clippy/fmt/test
- 依赖审计：OCR 无；ECC 可选 cargo audit/deny
- 输出形式：OCR 行级评论+JSON；ECC CRITICAL/HIGH/MEDIUM 报告
- 多语言统一：OCR 是；ECC 否 (Rust 专用)

三、共同覆盖的检查类别

- 错误处理：生产路径滥用 unwrap/expect/panic/todo
- unsafe：缺少安全说明、裸指针风险
- 所有权：不必要 clone、String/Vec 过度拥有
- 并发/async：async 中阻塞 I/O、Send/Sync 边界
- 安全：SQL/命令注入、硬编码密钥、路径遍历
- 性能：热路径多余分配、循环内重复创建
- 模式匹配：业务枚举滥用通配符、非穷尽匹配
- 代码卫生：死代码、Clippy allow 滥用

四、OCR 更强领域

- 所有权进阶：RefCell 误用、Rc/Arc 循环引用需 Weak
- unsafe 细节：static mut、transmute、MaybeUninit、FFI 边界
- 并发细节：锁跨 await、原子序、check-then-act 竞态
- async 进阶：cancellation safety、JoinHandle 丢弃、重试无 backoff
- API 设计：newtype、typed ID、From/TryFrom/AsRef trait 边界
- Cargo 工程：独立 cargo_toml.md (edition、MSRV、feature、发布元数据)

五、ECC 更强领域

- 工具链门禁：必须先通过 check/clippy/fmt/test
- 依赖安全：cargo audit / cargo deny
- 流程规范：CRITICAL/HIGH/MEDIUM + Approve/Warning/Block
- 结构指标：函数大于 50 行、嵌套大于 4 层
- 生态约定：thiserror/anyhow、public API 文档
- 并发工程：Mutex poison、死锁模式、无界 channel 需说明

六、优劣势总结

6.1 OCR 优势

- 按文件类型自动匹配，无需额外命令
- unsafe/async/FFI 等高级语义覆盖更深
- 多语言统一审查流水线，输出可落地到 PR 行级评论
- Cargo.toml 专项规则完整
- 不依赖本地 Rust 构建环境即可做 diff 审查

6.2 OCR 劣势

- 无 cargo check/clippy/test 硬门禁
- 无严重级别与审批标准
- 无 cargo audit 供应链漏洞检测
- 缺少 thiserror/anyhow、文档、函数长度等工程规范
- 纯 prompt 驱动，执行稳定性依赖模型

6.3 ECC 优势

- 先跑工具链再审查，减少无效语义分析
- Clippy/fmt/test/audit 形成确定性第一层过滤
- 严重级别清晰，适合 Block/Warning/Approve 流程
- Mutex poison、死锁、channel 等工程经验丰富
- rust-patterns skill 示例丰富，适合 onboarding

6.4 ECC 劣势

- 必须本地可构建，对 snippet/跨 crate 场景不友好
- Rust 专用，难融入多语言统一审查
- cancellation、原子序、FFI 等高级语义覆盖不如 OCR
- 无行级评论定位体系
- 依赖 Claude Code 生态 (Bash/Read/Grep)

七、关键差异对照

- cargo check/clippy/fmt/test: OCR 无, ECC 有
- cargo audit/deny: OCR 无, ECC 有
- 严重级别/审批标准: OCR 无, ECC 有
- unsafe 深度: OCR 强, ECC 中
- async cancellation: OCR 有, ECC 弱
- Mutex poison/deadlock: OCR 无, ECC 有
- thiserror/anyhow: OCR 无, ECC 有
- public API 文档: OCR 弱, ECC 强
- Cargo.toml 规则: OCR 强, ECC 弱
- 行级评论定位: OCR 有, ECC 无
- 多语言统一审查: OCR 有, ECC 无

八、推荐组合方案

- 第1层 — ECC: cargo check/clippy/fmt/test/audit 作为硬门禁
- 第2层 — OCR: rust.md 做 unsafe、async、FFI、API 设计等语义审查

- 第3层 — OCR: cargo_toml.md 审查 manifest、feature、MSRV、发布元数据
- 第4层 — ECC: 严重级别、文档规范、函数长度等团队流程规则

九、场景选型建议

- 多语言 PR 自动审查、行级评论、低误报 -> OCR
- Rust 单体/库项目深度审查、必须先绿 CI -> ECC
- unsafe/async/FFI 高风险代码 -> OCR 规则更深
- 团队规范、文档、Clippy、依赖审计 -> ECC 更完整
- 只改几行 Rust、无完整构建环境 -> OCR
- Rust 项目 onboarding / 教学 -> ECC + rust-patterns

十、参考资料

- OCR: internal/config/rules/rule_docs/rust.md
- OCR: internal/config/rules/rule_docs/cargo_toml.md
- ECC: github.com/affaan-m/everything-claude-code/agents/rust-reviewer.md
- ECC: github.com/affaan-m/everything-claude-code/commands/rust-review.md
- ECC: github.com/affaan-m/everything-claude-code/skills/rust-patterns/SKILL.md